

Domain 3: Claude Code Configuration & Workflows

Domain 3 makes up 20% of the scored content, and it's where you learn to shape Claude Code into a dependable part of your engineering workflow. You'll cover how to configure Claude's behavior through CLAUDE.md and Agent Skills so it follows your project's conventions, how to drive work with plan mode and slash commands, how to build agentic workflows that carry a task from start to finish, and how to wire Claude Code into your CI/CD pipeline so it pulls its weight in your automation, too.

Think of configuration as the difference between Claude Code guessing at your project and actually knowing it. Skip it, and you get inconsistent behavior, repeated corrections, and an assistant that works against your conventions instead of with them. Set it up well, and Claude Code shows up already knowing how your codebase works and what you expect from it. So, where does all that configuration actually live?

A. The CLAUDE.md Configuration Hierarchy

CLAUDE.md is the primary way to give Claude Code persistent, project-level instructions. It is a Markdown file that Claude reads at the start of every session to understand your project's conventions, coding standards, architecture decisions, and preferences. Think of it as a README written specifically for Claude rather than for human developers. Anthropic's documentation describes CLAUDE.md as the place for "important project information, conventions, and frequently used commands."

Claude Code supports a hierarchy of CLAUDE.md files at three levels: project root, subdirectory, and user home, plus a modular `@imports` mechanism for sharing standards across packages. Understanding which level to use and recognizing that all CLAUDE.md content is advisory (not guaranteed) are two of the most frequently tested concepts in Domain 3.

Every Claude Code session begins by reading applicable CLAUDE.md files and injecting their contents into the system prompt. The memory system merges all applicable files at session start, global user preferences combine with project-specific context. This means the root CLAUDE.md, any applicable subdirectory CLAUDE.md files, and the user-level CLAUDE.md are all loaded together, giving Claude a composite view of the project and the developer's preferences.

Root CLAUDE.md

The root CLAUDE.md sits at the top of your project directory. It contains project-wide instructions — coding standards, architectural patterns, and general conventions that apply everywhere in the codebase. Every session in the project loads this file.

Common contents include: the programming language and framework being used, preferred design patterns, naming conventions, testing requirements, build commands, and descriptions of intentional patterns that might otherwise look like anti-patterns (like force-unwrapping optionals in test files or using large coordinator classes that follow an established architecture).

A well-structured root CLAUDE.md helps Claude answer questions like: What language and framework is this project using? What are the naming conventions? Which patterns are intentional and should not be

flagged? What commands should Claude run for testing? What modules or files should Claude avoid modifying?

It should contain information that materially changes Claude's decisions, not information Claude can easily infer from reading the code. Brief, explicit, and high-signal content performs better than long, verbose instructions. Anthropic's own guidance recommends keeping this section concise, if you have more than 15 rules, you likely haven't identified which rules are genuinely load-bearing.

Subdirectory CLAUDE.md Files

CLAUDE.md files can be placed inside any folder in your project. When Claude is working on files within that directory, it loads that directory's CLAUDE.md in addition to the root file. This lets you scope instructions to specific areas of your codebase.

For example, you might place a CLAUDE.md inside `/terraform/` with Terraform-specific conventions, while `/kubernetes/` has its own CLAUDE.md with Kubernetes-specific guidance. This avoids mixing unrelated conventions in the root file and ensures Claude only receives context relevant to the area it's working on.

The directory-specific file supplements, rather than replaces, the root CLAUDE.md. Both are loaded together when Claude works in that directory. If there is a conflict between the root and subdirectory instructions, Claude may not resolve it consistently, which is one reason contradictory instructions should be avoided.

User-Level CLAUDE.md

The user-level CLAUDE.md at `~/.claude/CLAUDE.md` applies across all projects for a specific developer. This is where personal preferences go, things such as your preferred coding style quirks, communication preferences, or cross-project tooling knowledge that you want Claude to follow regardless of which project you are working on.

This file is local to your machine and is never committed to version control. It merges with the project-level files at session start without conflicts, personal preferences layer on top of project conventions.

EXAM TIP: The exam frequently tests whether you can identify which CLAUDE.md file is appropriate for a given scenario. If the instruction applies to the entire team and should be version-controlled, it goes in the project's root CLAUDE.md. If it's a personal preference, it belongs in `~/.claude/CLAUDE.md`. If it applies only to a specific part of the codebase, consider a subdirectory CLAUDE.md or path-scoped rules.

@imports for Shared Standards

CLAUDE.md supports `@imports` a syntax that lets you reference external Markdown files from within your CLAUDE.md. Instead of duplicating content across multiple files, you can import shared standards from a central location. This is particularly valuable in monorepo architectures where multiple packages need overlapping but not identical sets of standards.

For example, if your monorepo has shared coding standards stored in `/docs/standards/`, each package's CLAUDE.md can selectively import only the relevant standards:

```
# /packages/auth/CLAUDE.md
@docs/standards/security-rules.md
@docs/standards/testing-patterns.md

# /packages/notifications/CLAUDE.md
@docs/standards/testing-patterns.md
```

The auth package imports security rules because it handles user data, while the notifications package only needs the general testing patterns. Each package maintainer decides which standards are relevant based on their domain knowledge.

The key advantage of `@imports` over duplicating standards across packages is maintainability. When the security rules change, you update one file and every package that imports it gets the update. Without `@imports`, you would need to manually update every package's CLAUDE.md individually.

EXAM TIP: When a question describes a monorepo where package maintainers understand their own domain requirements and the question asks how to avoid duplicating irrelevant standards across packages, `@imports` in each package's CLAUDE.md is typically the right answer. This relies on each maintainer knowing which standards apply to their package. Compare this with `.claude/rules/` which uses path patterns to auto-load rules. That approach doesn't require maintainer knowledge but requires someone to explicitly list every package directory in the YAML frontmatter.

Advisory Nature of CLAUDE.md

One critical characteristic of CLAUDE.md that the exam tests repeatedly is that it is advisory, not deterministic. Claude processes the instructions as context, but it does not guarantee 100% compliance. Anthropic's documentation states that Claude treats CLAUDE.md content as context that "may or may not be relevant" to the current task. The memory system influences behavior, but it is not a hard policy engine.

In practice, this means Claude usually follows CLAUDE.md instructions but not always. Adding emphasis markers such as "IMPORTANT" or "YOU MUST" can improve adherence. Anthropic's own documentation confirms that marking a rule as IMPORTANT increases the probability that Claude treats it as a priority constraint rather than a preference. However, even with emphasis, compliance is probabilistic, not guaranteed.

For rules you need Claude to follow every single time without exception, you need a deterministic enforcement mechanism like hooks or permission settings. This distinction between advisory and deterministic is the most frequently tested concept across all of Domain 3.

The /memory Diagnostic Command

The `/memory` command shows which memory files (including CLAUDE.md) are currently loaded in Claude's context. This is the most efficient first diagnostic step when Claude inconsistently follows conventions defined in CLAUDE.md.

If your CLAUDE.md specifies that endpoint handlers should use a custom ApiError class but Claude sometimes uses generic try/catch blocks, the first step is to run `/memory` and verify that your

CLAUDE.md is actually being loaded. If it's not loaded, that explains the inconsistency, perhaps the file is in the wrong location, has a filename typo, or the directory scope is incorrect. If it is loaded, the issue is the advisory nature of CLAUDE.md, and you may need stronger enforcement through hooks.

EXAM TIP: When a question describes inconsistent convention-following "across different coding sessions" and asks for the "most efficient first diagnostic step", the answer is `/memory`. Don't jump to adding more examples, creating path-specific rules, or searching for conflicting instructions until you've first confirmed the file is loading.

Common Mistakes

- Treating CLAUDE.md as a hard enforcement mechanism rather than advisory guidance.
- Placing personal preferences in the project root CLAUDE.md instead of `~/.claude/CLAUDE.md`.
- Making the root CLAUDE.md too long (300+ lines) with information Claude can infer from reading the code.
- Contradicting instructions between root and subdirectory files, which creates inconsistent behavior.
- Not verifying with `/memory` that CLAUDE.md is actually loading before adding stronger enforcement.

References

- <https://docs.anthropic.com/en/docs/claude-code/memory>
- <https://docs.anthropic.com/en/docs/claude-code/settings>

B. Path-Scoped Rules with `.claude/rules/`

The `.claude/rules/` directory provides a more targeted alternative to subdirectory CLAUDE.md files. Files placed here use YAML frontmatter to specify exactly which file paths they apply to. When Claude works on a file matching the specified path pattern, the corresponding rule file loads automatically into context. When Claude is working on unrelated files, the rule stays out of context entirely, saving tokens and avoiding irrelevant instructions.

Rules files are version-controlled alongside the project and follow the same advisory nature as CLAUDE.md, they influence Claude's behavior but do not deterministically enforce it. The primary advantage over root CLAUDE.md is token efficiency: only relevant rules load for each file type, rather than every instruction loading for every task.

YAML Frontmatter Path Patterns

A rules file looks like this:

```
---
paths:
  - "terraform/**/*"
---
```

Use Terraform 1.5+ syntax. Always include a description for each variable. Pin provider versions in `required_providers` blocks.

This rule loads only when Claude edits files under the `terraform/` directory. When Claude is working on Kubernetes manifests or CI/CD scripts, this rule stays out of context entirely.

You can create multiple rules files, each targeting different path patterns. For example:

```
.claude/rules/terraform.md → paths: ["terraform/**/*"]
.claude/rules/kubernetes.md → paths: ["kubernetes/**/*"]
.claude/rules/typescript.md → paths: ["**/*.ts", "**/*.tsx"]
.claude/rules/pipelines.md → paths: ["pipelines/**/*", ".github/**/*"]
```

Each file carries its own domain-specific conventions, and Claude only loads the ones relevant to the files it's currently working on.

When to Use Rules vs. CLAUDE.md

Mechanism	Best For	Token Behavior	Version Controlled
Root CLAUDE.md	Project-wide instructions that apply everywhere	Always loaded	Yes
Subdirectory CLAUDE.md	Simple, directory-scoped conventions	Loaded when working in that directory	Yes
.claude/rules/	Domain-specific conventions for distinct file types	Loaded only when matching files are touched	Yes

Use `.claude/rules/` files when your project has distinct areas with different conventions and you want to minimize token consumption. A 500-line root CLAUDE.md that mixes Terraform, Kubernetes, and CI/CD conventions wastes tokens every time Claude works on just one of those areas. Path-scoped rules solve this by loading only what's relevant.

Use subdirectory CLAUDE.md files when the conventions are simpler and the directory structure already provides natural scoping. Use root CLAUDE.md for project-wide instructions that apply everywhere.

EXAM TIP: The exam tests path-scoped rules in scenarios involving infrastructure-as-code repositories or monorepos with distinct file types. If a question mentions that a root CLAUDE.md has grown too large (e.g., 500+ lines) and irrelevant rules are consuming tokens when working on specific file types, the answer is `.claude/rules/` files with YAML frontmatter path scoping.

Common Mistakes

- Using `.claude/rules/` for instructions that should apply everywhere, those belong in root CLAUDE.md.
- Assuming rules files provide deterministic enforcement, they are advisory, like CLAUDE.md.
- Creating overlapping path patterns across multiple rule files, which can cause conflicting instructions.

Reference

- <https://docs.anthropic.com/en/docs/claude-code/settings>

C. Custom Slash Commands and Skills

Claude Code skills are reusable workflow definitions that developers can invoke on demand via /slash commands. They solve the problem of repetitive workflows that need to run the same way every time without cluttering the always-loaded CLAUDE.md with instructions for tasks that only happen occasionally.

A key insight is that skills are on-demand and they only load when explicitly invoked by a developer typing the slash command. This contrasts with CLAUDE.md (always loaded) and rules (conditionally loaded based on file paths). Skills are the right choice when instructions are needed occasionally for specific workflows, not as persistent context.

Skill File Structure

A skill is defined in a SKILL.md file placed inside `.claude/skills/<skill-name>/`. The directory name determines the slash command. For example:

```
.claude/skills/migrate-component/SKILL.md
```

This creates a `/migrate-component` slash command. The SKILL.md file contains the step-by-step instructions for Claude to follow during the workflow. The content is plain Markdown — it can include checklists, code examples, decision trees, and any other instructions that define the workflow.

A typical skill file might look like:

```
Component Migration Workflow
1. Read the source component and identify all props and state
2. Create the new component file following our naming convention
3. Migrate props to TypeScript interfaces
4. Convert class lifecycle methods to hooks
5. Update all import statements in consuming files
6. Run the test suite and fix any failures
7. Update the component's Storybook story
```

Project-Level vs. User-Level Skills

Project-level skills are placed in `.claude/skills/` at the project root and committed to version control. Every team member gets the same skill, and updates propagate through Git. This is the correct choice when the skill is a team workflow that should stay in sync as the team iterates on it.

User-level skills are placed in `~/claude/skills/` on each developer's machine. These are personal workflows that only one developer uses. They are not committed to version control and are not shared with the team.

EXAM TIP: The exam tests skill placement in scenarios where a team needs a shared, version-controlled workflow invoked by a slash command (like `/migrate-component` or `/review`). The correct answer is the project-level path: `.claude/skills/<name>/SKILL.md` at the project root, committed to version control. Watch for distractors like `~/.claude/skills/` (user-level, not shared), `settings.json` (not how skills work), or root `CLAUDE.md` (loads every session, not on demand).

Skills vs. CLAUDE.md vs. Rules

The distinction is about when instructions load:

Mechanism	When It Loads	Use For	Token Impact
CLAUDE.md	Every session, always in context	Conventions that should always apply	Always consumed
Rules	Conditionally, when Claude works on matching file paths	Path-specific conventions	Consumed only when files match
Skills	On demand, when a developer invokes the slash command	Complex workflows that run occasionally	Consumed only when invoked

WORKED EXAMPLE:

A team has an 8-item code review checklist that they use when reviewing PRs. They don't use this checklist during feature development, debugging, or documentation work. If the checklist were in `CLAUDE.md`, it would load during every session, consuming tokens during tasks where it provides no value. If it were in `.claude/rules/`, it would need a path pattern, but code reviews apply to all file types, so the pattern would be `**/*`, which loads it everywhere, no better than `CLAUDE.md`. The correct placement is a skill: `.claude/skills/review/SKILL.md`. Developers invoke `/review` only when they need the checklist, keeping it out of context during all other work.

Common Mistakes

- Putting detailed workflow instructions in `CLAUDE.md` when they're only used occasionally, use a skill instead.
- Using `~/.claude/skills/` for team workflows that should be shared and version-controlled.
- Confusing skills (on-demand) with rules (auto-loaded based on file paths).

Reference

- <https://docs.anthropic.com/en/docs/claude-code/skills>

D. Hooks for Deterministic Enforcement

Hooks are the mechanism for deterministic enforcement in Claude Code. Unlike `CLAUDE.md` instructions, which are advisory, hooks are user-defined shell commands that run automatically at specific points in Claude's execution life cycle. They always execute, Claude cannot skip or ignore them. This makes hooks the correct choice when a rule must be followed every single time without exception.

Anthropic's documentation describes hooks as "user-defined event handlers that run shell commands or scripts at specific points in Claude Code's lifecycle." The critical distinction is that hooks do not depend on the model remembering to format code or run tests, they execute every single time their conditions are met, regardless of what the AI decides to do.

Hooks live in JSON configuration files: `.claude/settings.json` (committed to git, shared with the team) or `.claude/settings.local.json` (gitignored, personal). Both files are picked up automatically by the Claude Code CLI.

PreToolUse Hooks

PreToolUse hooks run before Claude executes a tool (like Edit, Write, Bash, or any MCP tool). They can inspect the planned action and either allow it, block it, or modify it. When a PreToolUse hook returns exit code 2, the tool is blocked, even in `bypassPermissions` mode or with `--dangerously-skip-permissions`. This makes PreToolUse hooks the strongest enforcement mechanism available.

Use PreToolUse hooks when you need to prevent Claude from doing something, like blocking destructive shell commands (`rm -rf, DROP TABLE`), validating inputs before they execute, or enforcing that certain files are never modified.

PostToolUse Hooks

PostToolUse hooks run after Claude has completed a tool execution successfully. They receive both the tool input (arguments sent to the tool) and the tool response (result it returned). They can inspect the result and run follow-up actions.

Use PostToolUse hooks when you need to automatically process Claude's output, like running a code formatter on every file Claude modifies, executing lint checks after code changes, or logging tool invocations for audit purposes.

A common PostToolUse configuration runs Black (Python) or Prettier (JavaScript/TypeScript) after every file edit:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write\n\"$CLAUDE_TOOL_INPUT_FILE_PATH\""
          }
        ]
      }
    ]
  }
}
```


Hook Matchers

Each hook specifies a matcher that determines which tool triggers the hook. The matcher is an optional regex filter. Common matchers include:

- `Edit` or `Write`: triggers on file modifications
- `Edit|Write`: triggers on either edit or write operations
- `Bash`: triggers on shell command execution
- `Write(*.py)`: triggers only on Python file writes
- Specific MCP tool names using the pattern `mcp_server_tool`: triggers on calls to external tools

The matcher is what makes hooks targeted rather than blanket. You can have a `PostToolUse` hook that runs Prettier only on file edits, without triggering on Bash commands or MCP calls. If no matcher is specified, the hook fires for every tool call.

Hooks vs. CLAUDE.md — Advisory vs. Deterministic

This is the most frequently tested distinction in Domain 3.

Concept	Best Used For	Key Benefit	Common Exam Trap
CLAUDE.md (advisory)	Coding preferences, architectural guidelines, style conventions	Easy to write, always in context	Trusted to guarantee compliance when it cannot
Hooks (deterministic)	Formatting enforcement, validation, security checks, logging	Always runs, no exceptions	Overlooked when a CLAUDE.md instruction is inconsistently followed

WORKED EXAMPLE:

A CLAUDE.md file includes the rule: "IMPORTANT: Always run Prettier after editing TypeScript files." Despite the emphasis, approximately 15% of files Claude generates still have inconsistent formatting. Adding stronger language does not eliminate the remaining violations. The solution is a `PostToolUse` hook with an `Edit|Write` matcher that automatically runs Prettier on every modified file. The hook runs outside of Claude's decision-making process — Claude does not decide whether to format; the hook handles it deterministically after every file modification. Result: 100% formatting compliance, regardless of what Claude generates.

EXAM TIP: When a question describes Claude inconsistently following a CLAUDE.md formatting rule, and emphasis doesn't eliminate the problem, the answer is a `PostToolUse` hook. The exam specifically tests whether you understand that CLAUDE.md is advisory while hooks are deterministic. The most common scenario is code formatting: Prettier, Black, or similar tools should run on every file modification.

Common Mistakes

- Expecting CLAUDE.md emphasis ("IMPORTANT," "YOU MUST") to guarantee compliance improves probability but cannot eliminate violations.
- Using a `PreToolUse` hook for post-processing (like formatting) — that's the job of `PostToolUse`.
- Forgetting that hooks fire for subagent actions too, if Claude spawns a subagent, your hooks execute for every tool the subagent uses.

- Not specifying a matcher, which causes the hook to fire on every tool call, adding latency to operations that don't need it.

Reference

- <https://docs.anthropic.com/en/docs/claude-code/hooks>

E. Permissions and Access Controls

Permissions provide hard guardrails that prevent Claude from performing specific actions entirely. Unlike CLAUDE.md (advisory) and hooks (run scripts around actions), permissions block actions before they can even begin. Permissions are the most restrictive enforcement mechanism; they are a hard block that Claude physically cannot circumvent.

Together, permissions, CLAUDE.md, and hooks form a spectrum of enforcement strength. The exam frequently tests whether you can match each requirement to the appropriate mechanism based on how strictly it needs to be enforced.

permissions.deny for Hard Restrictions

The `permissions.deny` setting in project or user settings completely blocks Claude from performing specific tool actions on matching paths. For example:

```
{
  "permissions": {
    "deny": ["Edit(./db/migrations/**)"]
  }
}
```

This prevents Claude from editing any file in the `db/migrations/` directory. Unlike a CLAUDE.md instruction saying "don't edit migrations," this is a hard block, so Claude physically cannot modify those files. Unlike a PreToolUse hook, this doesn't require writing a script, it's a declarative configuration.

Combining Permissions, CLAUDE.md, and Hooks

Different requirements call for different enforcement mechanisms. A common exam scenario presents three requirements that each need a different mechanism:

Requirement	Enforcement Level	Mechanism
"Claude must never modify files in db/migrations/"	Hard block	permissions.deny
"Claude should prefer a custom logging module over console.log"	Advisory preference	CLAUDE.md
"All TypeScript files must be auto-formatted with Prettier after every edit"	Deterministic automation	PostToolUse hook

The key insight: the word "never" signals `permissions.deny`. The words "prefer" or "should" signal `CLAUDE.md`. The word "automatically" or "always" with an automation task signals hooks.

EXAM TIP: When a question presents multiple requirements and asks you to "restructure" them across Claude Code's configuration mechanisms, match each requirement's enforcement level: permissions for absolute blocks, `CLAUDE.md` for preferences and conventions, and hooks for automatic actions.

WORKED EXAMPLE:

A team currently has all three requirements in their `CLAUDE.md`:

1. "Never modify database migration files"
2. "Use our custom ErrorHandler class instead of generic try/catch"
3. "Run ESLint after every TypeScript file modification"

All three are in `CLAUDE.md`, but only requirement 2 belongs there. Requirement 1 needs `permissions.deny` because "never" means absolute block. Requirement 3 needs a `PostToolUse` hook because "run after every modification" means deterministic automation. After restructuring: `permissions.deny: ["Edit(./db/migrations/**)"]`, `CLAUDE.md` retains only the `ErrorHandler` preference, and a `PostToolUse` hook with `Edit|Write` matcher runs ESLint on `.ts` files.

Reference

- <https://docs.anthropic.com/en/docs/claude-code/settings>

F. MCP Server Configuration in Claude Code

The Model Context Protocol (MCP) allows Claude Code to connect to external tools, data sources, and services. MCP servers extend Claude's capabilities beyond its built-in tools. Configuration scoping determines which servers are shared with the team and which are personal.

MCP servers are added via the CLI command `claude mcp add <server-name>` or by editing the configuration files directly. By default, MCP servers are added at project scope. Environment variables can be used for secrets (like API keys) through expansion syntax, so credentials are never committed to version control.

.mcp.json — Project-Level Configuration

The `.mcp.json` file lives at the project root and is committed to version control. All team members who work on the project get the same MCP servers. Use this for shared tools that the entire team needs — like a venue lookup service, a database explorer, or an internal API.

Environment variables are expanded in the configuration, allowing you to reference secrets without committing them:

```
{
  "mcpServers": {
    "venue-lookup": {
      "command": "node",
      "args": ["./.mcp-servers/venue-lookup/index.js"],
```

```
    "env": {  
      "API_KEY": "${VENUE_API_KEY}"  
    }  
  }  
}
```

The `${VENUE_API_KEY}` is expanded from each developer's environment at runtime. The API key never appears in version control.

~/.claude.json — User-Level Configuration

The `~/.claude.json` file lives in the user's home directory and is not shared with the team. Use this for personal or experimental MCP servers that only you are testing.

EXAM TIP: The exam tests MCP scoping with scenarios like "Add a shared venue lookup server for the team AND a personal experimental playlist server for yourself." Shared → `.mcp.json`. Personal → `~/.claude.json`. Watch for distractors that reverse the order.

MCP Prompts as Slash Commands

When an MCP server exposes prompts (like `deploy_checklist` or `incident_response`), Claude Code surfaces these as slash commands with the naming convention `/mcp_<servername>_<promptname>`. Arguments are passed after the command name.

These prompts are not automatically prepended to conversations, not added to the tool registry for automatic invocation, and not surfaced as @-mentionable resources. They are strictly slash commands that developers invoke explicitly.

EXAM TIP: If the exam asks how MCP prompts become accessible within Claude Code the answer is slash commands. Every other option is a distractor.

MCP Resources for Content Catalogs

MCP servers can also expose resources structured content catalogs that give Claude visibility into what data each server contains. This is valuable when Claude connects to multiple servers (like an issue tracker, a documentation wiki, and a database explorer) and needs to efficiently query across them.

Without resources, Claude has no visibility into what content each server contains, leading to excessive exploratory tool calls and wasted context. When each server exposes its content catalog as MCP resources (issue summaries, documentation hierarchy, database schemas), Claude can make informed decisions about which server to query for each part of a cross-system question.

The difference between resources and tools is important: tools are callable functions that perform actions (query a database, create a ticket). Resources are read-only content catalogs that describe what data each server contains. Resources help Claude decide which tool to call, they are informational, not operational.

EXAM TIP: When a question describes Claude making too many sequential exploratory calls across multiple MCP servers because it "lacks visibility into what content each server contains," the answer is to

expose each server's content catalog as MCP resources.

Common Mistakes

- Putting personal MCP servers in `.mcp.json` (project-level). They'll be committed to version control and shared with the team.
- Hardcoding API keys in `.mcp.json` instead of using environment variable expansion.
- Confusing MCP resources (read-only content catalogs) with MCP tools (callable functions).
- Expecting MCP prompts to auto-load into context, they only appear as slash commands.

References

- <https://docs.anthropic.com/en/docs/claude-code/settings>
 - <https://docs.anthropic.com/en/docs/agents-and-tools/mcp>
-

G. Plan Mode vs. Direct Execution

Claude Code operates in two primary modes that serve different purposes. Plan mode is Claude's "think before acting" mode where Claude analyzes the codebase, evaluates tradeoffs, and presents a plan without making changes. Direct execution is Claude's "just do it" mode, where Claude reads code, makes changes, runs tests, and delivers the result. Knowing when to use each is a significant portion of the Domain 3 exam.

The decision between modes is not about preference or difficulty; it is about whether the task benefits from explicit analysis before action. Some tasks have a single obvious path; others require investigation, enumeration, and comparison before committing. The exam tests your judgment in distinguishing these two cases.

When to Use Plan Mode

Use plan mode when:

- You are unfamiliar with the module or codebase area you need to work in.
- A critical production bug needs investigation before attempting a fix, you need to understand the module's architecture, enumerate potential root causes, and prioritize fixes systematically.
- The task has multiple possible approaches, and you need to evaluate tradeoffs before committing (like choosing between WebSockets, Server-Sent Events, or polling).
- A security audit or large migration requires mapping affected code paths across many files before implementing.

WORKED EXAMPLE:

A developer is assigned a critical production bug in the inventory service. They have never worked on this module before. The bug causes inventory counts to drift from the database under high concurrency. Without plan mode, the developer might ask Claude to "fix the concurrency bug" — but Claude would be guessing at the architecture, the locking strategy, and the root cause. With plan mode, Claude first reads the module's structure, identifies the database access patterns, maps the locking mechanisms, enumerates potential race conditions, and presents a prioritized list of likely root causes. The developer reviews the analysis before asking Claude to implement a fix.

When to Use Direct Execution

Use direct execution when:

- The task is simple, well-scoped, and low-risk, like adding a date validation check to one function in one file.
- The requirements are clear, and you know exactly what needs to happen.
- The scope is limited, one file, one function, one change.

Scenario	Right Mode	Reason
Add a date validation check to one function	Direct execution	Simple, one file, one change
Critical bug in unfamiliar module	Plan mode	Need to understand architecture first
Choose between WebSockets vs SSE vs polling	Plan mode	Multiple approaches, need tradeoff analysis
Rename getUserData to fetchUserProfile everywhere	Direct execution	Mechanical, clear outcome
Library migration across 45 files	Plan mode	Large scope, multiple decisions
Add a null check to one conditional	Direct execution	Simple, well-scoped
Improve error handling across a module	Plan mode	Ambiguous, many decisions, interacting concerns

When Multi-Phase Workflows Improve Outcomes

Not every task benefits from a multi-phase approach (analyze → propose → implement with review).

Benefits from multi-phase: Tasks that are ambiguous, complex, or have multiple valid approaches. For example, "Improve unambiguous error handling throughout the data processing module, add try/catch blocks, provide meaningful error messages, and ensure failures don't silently corrupt data." This has many decisions to make, interacting concerns, and no single obvious path.

Does NOT benefit from multi-phase: mechanical, well-defined tasks with a clear, correct outcome. For example, "rename the `getUserData` function to `fetchUserProfile` everywhere it's used." This is an unambiguous find-and-replace, multi-phase planning adds overhead without improving the result.

EXAM TIP: When a question asks which of two requests benefits more from an explicit multi-phase workflow, it's always the ambiguous, judgment-heavy task, not the mechanical transformation.

Reference

- <https://docs.anthropic.com/en/docs/claude-code/best-practices>

H. Iterative Development Workflows

The exam tests your ability to choose the right iterative workflow pattern for each situation. The key insight is that different types of problems require different iteration strategies, test-driven for complex algorithms, incremental for interacting issues, concrete examples for edge cases, and requirements discovery for undefined problems.

Iterative development with Claude Code relies on the quality of your feedback. What moves it forward is a clear, unambiguous signal about what needs to change, so vague nudges like "try again" or "do better" tend to leave Claude guessing. The feedback that lands is concrete: test results, error messages, specific input/output pairs. Subjective coaching like "handle edge cases better" or "be more careful" gives Claude nothing solid to act on.

Test-Driven Iteration

The most effective pattern for iterative refinement with Claude Code is to write tests first, then ask Claude to write code that passes them. When tests fail, share the failure output with Claude and ask it to fix the code. This creates a tight feedback loop where test results serve as objective, unambiguous signals about what needs to change.

This approach works particularly well for complex algorithms with specific edge cases and performance requirements (like graph traversal with cycles, disconnected nodes, and weighted edges). Instead of describing desired behavior in natural language (which can be ambiguous), the tests define it precisely.

WORKED EXAMPLE:

A developer needs to implement a graph traversal algorithm that handles cycles, disconnected nodes, and weighted edges with specific performance constraints. Instead of describing these requirements in natural language:

1. Write a test suite with specific inputs and expected outputs for each edge case.
2. Ask Claude to implement the algorithm.
3. Run the tests and share the failure output: "Test 3 failed: expected [A, B, D] but got [A, B, C, D] for graph with cycle at node C."
4. Claude adjusts the implementation to handle the cycle detection.
5. Repeat until all tests pass.

Each iteration provides Claude with an objective, specific signal and not "handle cycles better" but "this specific input produced this wrong output."

EXAM TIP: When a question describes implementing a complex feature with specific requirements and asks how to structure the workflow for "efficient iterative refinement," the answer is test-driven iteration: write the test suite first, then iterate by sharing test failures.

Incremental Problem Solving

When multiple issues interact, like table column widths, date formatting, and page breaks in a PDF report, the correct approach is to fix them one at a time, testing after each change. Start with the most foundational issue (column widths), verify it works, then fix the next dependent issue (date formatting within the corrected columns), then the next (page breaks that depend on content height).

This is more reliable than batching all fixes into one prompt, because interacting problems create cascading side effects that are difficult to diagnose when everything changes simultaneously.

EXAM TIP: If a question describes interacting issues and asks for the "most effective approach for iterating toward a working solution," the answer is incremental fixes with testing after each step.

Providing Concrete Test Cases

When Claude's output doesn't handle edge cases correctly, the most effective iteration technique is providing a concrete test case with specific input and expected output. For example, if a data migration script doesn't handle null values correctly, provide a specific input record with null values and show exactly what the output should be.

This is more effective than describing the problem in natural language ("handle edge cases better"), adding emphasis ("IMPORTANT: think harder"), asking Claude to regenerate entirely, or manually editing the code yourself.

The reason concrete examples work better than descriptions is the same reason few-shot prompting works better than zero-shot: showing Claude the exact expected behavior removes ambiguity that descriptions leave open to interpretation.

Requirements Discovery Interview and TBD Patterns

When you have a rough idea but aren't sure about all the requirements for a robust implementation, two effective patterns exist:

Interview pattern: Ask Claude to interview you about the requirements before implementing, surfacing considerations you may not have thought of (like invalidation strategies, cache layers, consistency guarantees, and failure modes for a caching implementation).

TBD marker pattern: Write a specification with your known requirements and "TBD" markers for uncertain areas, then have Claude propose solutions for each TBD as it implements.

Both patterns help surface unknown unknowns before you commit to an implementation direction.

EXAM TIP: When a question describes someone with a rough idea ("Redis with 5-minute TTL") who is "new to production caching and isn't sure what other considerations a robust implementation requires," the answer involves either the interview or TBD approach, not jumping straight into a minimal implementation.

@References for One-Off Context

When you need Claude to follow patterns from existing code for a one-off task, use [@references](#) to include specific files directly in your prompt. For example, if you're implementing a payment processing module that should follow the same patterns as your existing [db_utils.py](#), [error_handlers.py](#), and [audit_logger.py](#), reference all three with [@](#) syntax.

This is preferable to describing the patterns in natural language (less precise), adding them to CLAUDE.md (unnecessary for a one-off task, they'd load every session), or asking Claude to explore the codebase to find the patterns itself (slower and less targeted).

EXAM TIP: When a question describes a one-off task where Claude should follow existing patterns in specific files, and the patterns are "well-documented in the team wiki and don't need additional project-level documentation," @references is the answer.

Common Mistakes

- Using vague feedback ("handle edge cases better") instead of concrete test cases with specific input and expected output.
- Batching multiple interacting fixes into one prompt instead of fixing them incrementally.
- Jumping straight to implementation when requirements are uncertain — the interview pattern surfaces critical considerations first.
- Adding patterns to CLAUDE.md for a one-off task when @references would keep the context temporary.

Reference

- <https://docs.anthropic.com/en/docs/claude-code/best-practices>
-

I. Session Management

Claude Code provides several mechanisms for managing sessions across work periods and parallel exploration paths. The exam tests your ability to choose the right session management strategy for each situation, from resuming named sessions to forking for parallel exploration to spawning sub-agents when context degrades.

Each mechanism addresses a different problem: resumption preserves accumulated context across time. Forking preserves context across parallel exploration paths. Sub-agent spawning addresses context degradation during long sessions. Scratchpad files address the gradual loss of specificity as conversation history grows. Understanding which problem each mechanism solves is the key to selecting the right one.

Resuming Named Sessions

The `--resume <session-name>` flag lets you continue a previous session by name. This preserves all accumulated context from the prior session. For example, if yesterday you named a session `auth-deep-dive` during a 2-hour investigation of authentication flows, you can resume it with `--resume auth-deep-dive`.

EXAM TIP: The exam may present a developer who worked on a specific investigation yesterday, has worked on other codebases since, and knows the session name. The answer is `--resume <name>`. Not `--continue` (picks up the most recent session, which might be from a different codebase). Not `--session-id` with a UUID (works but requires finding the transcript file). Not starting fresh (wastes all prior context).

Forking Sessions for Parallel Exploration

The `fork_session` feature creates branches from an existing session, preserving all accumulated context. This is essential when you need to explore two or more approaches independently after an initial analysis phase.

For example, after spending an hour analyzing a legacy authentication module and identifying two refactoring approaches (extracting a microservice vs. refactoring in-place), you can fork the session to explore each approach in its own branch. Each fork starts with the full analysis context but evolves independently. Neither approach contaminates the other's context.

EXAM TIP: When a developer needs to "independently explore two approaches in depth" after building a significant analysis context, `fork_session` is the answer. Starting fresh sessions (loses context) and exploring sequentially in the same session (approaches contaminate each other's context) are common distractors.

Resuming After Codebase Changes

When resuming a session after the underlying codebase changed (e.g., a teammate merged a PR overnight), the best approach is to do the following:

1. Resume the session to preserve accumulated context.
2. Inform Claude about which specific files changed.
3. Let Claude do targeted re-analysis of only the changed files.

This balances efficiency (don't re-read everything) with accuracy (don't work with a stale understanding of changed files).

EXAM TIP: Watch for the nuance in "3 of 12 files changed" scenarios. Don't start fresh (wastes context for the 9 unchanged files). Don't resume without mentioning changes (stale assumptions about the 3 changed files). Don't re-read all 12 files (wasteful). Resume and inform about the specific changes.

Scratchpad Files for Long Sessions

During extended exploration sessions (30+ minutes), Claude may start losing track of earlier findings. The agent might reference "typical rendering patterns" instead of the specific `VulkanPipeline` and `FrameGraph` classes it discovered earlier. This happens because older conversations gradually fade from active attention as the context window fills with newer content.

The most effective solution is having Claude maintain a scratchpad file, a persistent file on disk where Claude records key findings, architectural decisions, and important discoveries as it works. Unlike conversation history, which fades, a file on disk persists and can be re-read at any time.

EXAM TIP: When a question describes inconsistent answers about earlier findings in long sessions, scratchpad files are the answer. Not switching to a larger model (doesn't address the fundamental attention dilution issue). Not clearing context periodically (destroys accumulated knowledge). Not pre-generating file summaries (doesn't capture discoveries made during exploration).

Sub-Agent Spawning for Context Management

When an exploration session needs to pivot to a related but distinct area (e.g., from a rendering subsystem to physics integration), and you notice Claude losing specificity about earlier findings, you can:

1. Summarize key findings from the current exploration.
2. Spawn a sub-agent with that summary as initial context.

The sub-agent gets a fresh context window with the important findings preserved in compact form, plus full capacity to explore the new area. The main session retains its accumulated knowledge.

This pattern addresses context degradation without losing important findings. The summary acts as a compressed knowledge transfer between the parent session and the sub-agent.

The `/clear` Command

The `/clear` command resets Claude's conversation context entirely, starting a fresh session. Use this when accumulated context is actively harmful for example, when Claude has built up incorrect assumptions that you can't correct through conversation.

EXAM TIP: `/clear` is a last resort, not a first step. The exam typically presents `/clear` as a distractor there's almost always a better option that preserves useful context.

Session Situation	Right Approach	Wrong Approaches
Continue yesterday's named session	<code>--resume <name></code>	Start fresh, <code>--continue</code> (picks most recent, may be wrong project)
Explore two approaches independently	<code>fork_session</code>	Sequential in same session, start two fresh sessions
Long session losing specificity	Scratchpad file	Larger model, periodic <code>/clear</code> , pre-generate summaries
Pivoting to related area with degrading context	Sub-agent with summary	<code>/clear</code> and start over, ignore the degradation
Resume after 3 of 12 files changed	Resume + inform about changes	Start fresh, resume without mentioning changes
Accumulated incorrect assumptions	<code>/clear</code>	Continue with corrections (may not override assumptions)

References

- <https://docs.anthropic.com/en/docs/claude-code/sessions>
- <https://docs.anthropic.com/en/docs/claude-code/sub-agents>

J. Running Claude Code in CI/CD Pipelines

Claude Code can be invoked non-interactively in CI/CD pipelines for automated tasks like code reviews, test generation, release notes, and documentation updates. The `claude -p` flag (also `--print`) puts Claude in piped/programmatic mode, where it reads input from stdin or a prompt string and produces output without interactive prompts.

Understanding the flags that control behavior in CI environments is critical. Two flags in particular, `--system-prompt` vs. `--append-system-prompt`, are frequently tested because misusing them produces a specific, diagnosable failure pattern: Claude only comments on the piped diff and never reads surrounding files.

Non-Interactive Mode with `claude -p`

The `claude -p` flag puts Claude Code in piped/programmatic mode, it reads input from stdin or a prompt string and produces output without interactive prompts. This is the standard invocation for CI/CD pipelines, GitHub Actions, and other automated workflows.

In a GitHub Actions workflow, a typical invocation might look like:

```
echo "$PR_DIFF" | claude -p \  
  --append-system-prompt "Review for security issues and bugs. Report  
findings as JSON." \  
  --max-turns 10 \  
  --max-budget-usd 2.00 \  
  --output-format json
```

`--max-turns` and `--max-budget-usd` Flags

`--max-turns N` limits the number of agentic iterations Claude can perform in a single invocation. This prevents runaway loops where Claude keeps reading files, running tools, and iterating without converging.

`--max-budget-usd X` sets a hard dollar cap on how much a single invocation can spend on API tokens. This prevents expensive API calls from accumulating during large PR reviews.

Both flags are enforced by Claude Code itself (not the surrounding job runner), making them the correct mechanism when a question asks about enforcing per-invocation caps "within Claude Code" or "by Claude Code itself."

EXAM TIP: When a question describes expensive, long-running agentic loops on large PRs and asks how to enforce iteration and cost caps — the answer is `--max-turns N --max-budget-usd X` on the `claude -p` invocation. Distractors include `timeout-minutes` on the GitHub Actions step (job runner, not Claude Code), switching to a smaller model (doesn't cap totals), and `--permission-mode dontAsk` (controls permission prompts, not iterations or cost).

`--system-prompt` vs. `--append-system-prompt`

This distinction is critical and frequently tested.

`--system-prompt` completely replaces Claude Code's built-in system prompt with your custom instructions. Claude loses its default guidance for using file-reading tools, code navigation, and other built-in capabilities. Result: Claude only operates on what's piped to it — it won't read surrounding files, search the codebase, or use any of its built-in exploration tools.

`--append-system-prompt` adds your custom instructions to Claude Code's existing default prompt. Claude retains its full set of built-in tools and capabilities, plus your custom instructions on top.

Flag	Effect	Claude's Built-in Tools	Use When
<code>--system-prompt</code>	Replaces default prompt entirely	Lost: Claude won't use Read, Grep, Glob, etc.	You want full control and don't need built-in tools
<code>--append-system-prompt</code>	Adds to default prompt	Preserved: Claude retains all capabilities	You want Claude's built-in tools plus your custom instructions

EXAM TIP: When a question says Claude "only comments on the piped diff" but "never reads surrounding files" and the invocation uses `--system-prompt` the fix is to switch to `--append-system-prompt`. This preserves Claude's built-in tool-use guidance while adding your review instructions.

Permission Modes for CI

The `--permission-mode` flag controls how Claude handles permission prompts in non-interactive mode. The `dontAsk` mode auto-denies permission requests not explicitly allowed, which is appropriate for CI environments where there's no human to approve prompts.

Common Mistakes

- Using `--system-prompt` when you want Claude to read surrounding files this strips Claude's built-in tool guidance.
- Setting `timeout-minutes` on the GitHub Actions step instead of `--max-turns/--max-budget-usd` on the Claude invocation the job runner timeout kills the process without letting Claude finish cleanly.
- Confusing `--permission-mode dontAsk` (controls permissions) with `--max-turns` (controls iterations).

Reference

- <https://docs.anthropic.com/en/docs/claude-code/github-actions>

K. Automated Code Review Patterns

The CCA-F exam contains questions about building and optimizing automated code review pipelines. These questions test prompt design, output structure, noise reduction, and coverage improvement. This is one of the most heavily tested areas within Domain 3, with questions appearing in every exam scenario set.

The core challenge of automated code review is balancing precision (avoiding false positives) with recall (catching real bugs). Every prompt design decision shifts this balance. The exam tests whether you can identify the correct intervention for each type of imbalance.

Reducing False Positives with CLAUDE.md

When an automated review consistently flags patterns your team uses intentionally (force-unwrapping optionals in test files, large coordinator classes following your architecture, or importing internally-maintained modules marked deprecated in the public SDK), the solution is to document these accepted patterns in the project's CLAUDE.md.

This gives Claude persistent context about your project's conventions during every review, preventing it from generating findings on intentional patterns. The key insight is that these are not bugs, they are deliberate architectural decisions. Without project context, Claude reasonably flags them because they look like anti-patterns to a reviewer without domain knowledge.

Improving Recall with Few-Shot Examples

When reviews miss real bugs (low recall), adding few-shot examples that demonstrate the specific bug categories Claude should flag, race conditions, null dereferences, and error handling gaps, is the most effective approach. Few-shot examples teach Claude to recognize patterns by showing concrete instances of what to look for.

This is more effective than vague instructions to "be more thorough" (doesn't tell Claude what to look for), adding chain-of-thought prompts (helps reasoning but not pattern recognition), or expanding context with unrelated files (adds noise without signal).

EXAM TIP: When a question describes missed bugs and asks how to improve recall while maintaining precision, few-shot examples of the specific bug categories is the answer. Note the precision constraint: improving recall by making Claude flag everything would destroy precision. Few-shot examples improve recall on specific categories without the broad overcounting that general instructions produce.

Explicit Reporting Criteria

When too many findings are technically accurate but "not worth addressing" (minor style preferences, patterns acceptable in your codebase), the most effective prompt change before adding infrastructure complexity is to add explicit criteria defining which issues to report (bugs, security vulnerabilities) versus skip (minor style, local patterns).

This is the prompt-level equivalent of a bug filter. Instead of filtering after the fact, you prevent unwanted findings from being generated in the first place by defining the boundary between reportable and not-reportable.

Structured Output with Confidence and Severity Metadata

When a review prompt tells Claude to "only report high-confidence issues" and real bugs slip through undetected, the solution is to remove the suppressive filtering and instead instruct Claude to report all findings with a confidence level and severity tag, deferring filtering to a downstream processing step.

The problem with "only report high-confidence" is that Claude is often already confident in the very cases it gets wrong. Asking for more confidence does not filter out the wrong cases, it suppresses correct but uncertain findings while preserving confident but wrong ones.

Output Structure Design — The `detected_pattern` Field

When developers dismiss 35% of findings and you want to analyze what the system is getting wrong, the most useful addition to the output structure is a `detected_pattern` field recording the specific code construct that triggered each finding (e.g., "single-letter loop variable," "unused import," "missing null check").

This lets you analyze dismiss rates per detected pattern, identify which patterns systematically produce unhelpful findings, and adjust your prompts accordingly. This is more actionable than category-level analysis (too broad), confidence scores (poorly calibrated), or expanded descriptions (doesn't surface patterns).

Inline Reasoning with Findings

When the bottleneck is investigation time developers must click into each finding to read Claude's reasoning before deciding whether to address or dismiss. The solution is to require Claude to include its reasoning and confidence assessment inline with each finding.

This lets developers triage findings at a glance without clicking through to each one. It reduces investigation time without filtering findings before developer review.

Prior Review Context to Avoid Duplicates

When a review runs again after a developer pushes new commits addressing earlier findings, it may duplicate comments on already-fixed code. The solution is to include the prior review findings in context and instruct Claude to only report new or still-unaddressed issues.

This is more accurate than: post-processing filters (which use crude path/description matching), restricting scope to only newly modified files (misses regressions), or running reviews only at PR creation and pre-merge (skips intermediate feedback).

Severity Criteria with Concrete Examples

When severity ratings are inconsistent, the same pattern gets "critical" in one PR and "medium" in another. The solution is to include explicit severity criteria in your prompt with concrete code examples for each severity level.

Concrete examples calibrate Claude's judgment more effectively than: severity lookup tables in CLAUDE.md (advisory, still varies), relative severity within a PR (inconsistent across PRs), or asking Claude to include severity reasoning for manual calibration (doesn't solve the root inconsistency).

WORKED EXAMPLE:

Severity criteria with anchoring examples: Critical: SQL injection or unsanitized user input concatenated into a query string. Major: Logic bug that changes the output — e.g., off-by-one in loop boundary. Minor: Missing null check on optional data where the default behavior is safe. Skip: Naming style, import order, formatting, patterns documented as accepted in CLAUDE.md. Each tier has a concrete code example, not just a description. Claude can compare its findings against these examples to classify consistently.

Splitting Reviews into Focused Prompts

When a single review prompt covers multiple concern types (security, API design, business logic) and adding examples for one category hurts recall in another, the solution is to split the review into separate focused prompts, each with dedicated examples, then consolidate findings before posting.

This also applies when evaluation shows a recall tradeoff: improving business logic detection from 34% to 41% drops API design detection from 82% to 68%. Splitting into focused prompts eliminates this tradeoff because each prompt has its own examples and criteria.

Agentic Reviews for Cross-File Analysis

When reviews consistently miss bugs involving cross-file interactions (a PR renames function parameters but callers in unchanged files still use old argument names), the issue is that the review only sees the diff and changed files. The solution is to redesign the review as a turn-limited agentic task where Claude can read files and search the codebase via tools, following references to verify cross-file findings.

This is where `--append-system-prompt` becomes essential; you need Claude's built-in file-reading tools to be available, not just the piped diff.

Handling Truncated JSON Output

When a review using `tool_use` with a `report_findings` tool hits the `max_tokens` limit on a large PR, the JSON output gets truncated mid-structure, breaking the parser. The solution is to split the review into multiple API calls that each analyze a subset of the changed files, then merge the resulting findings arrays.

This is more robust than: increasing `max_tokens` (still might not be enough for very large PRs), retrying with "only critical findings" (loses coverage), or switching to markdown output (loses structured parsing).

Common Mistakes

- Using "be conservative" as a precision fix, it doesn't define the boundary and doesn't improve precision.
- Expecting confidence-based filtering to remove false positives, Claude is often confident on wrong findings.
- Running a single all-in-one review on 14+ files attention spreads thin, findings become inconsistent.
- Ignoring the `--system-prompt` vs `--append-system-prompt` distinction in CI review configuration.

References

- <https://docs.anthropic.com/en/docs/claude-code/best-practices>
- <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/increase-consistency>

L. Cost Optimization with the Message Batches API

The Message Batches API offers a 50% cost reduction compared to real-time API calls. However, results may take up to 24 hours to process. The key decision factor is always latency tolerance, whether anything is blocked waiting on the result. This section is closely related to Domain 4's batch processing coverage but focuses on the Claude Code-specific applications and exam patterns.

Batch vs. Real-Time Decision

The deciding question is always: does anything stop working while waiting for this result? If yes, use synchronous. If not, use a batch.

Workload	Right Choice	Reason
Overnight technical debt reports	Batch	Latency-tolerant, high volume, cost matters
End-of-week release notes (200 commits, ~12hr acceptable)	Batch	No urgency, significant cost saving
Pre-merge checks blocking developer workflow	Synchronous	Developer is waiting for the result
Real-time code review comments	Synchronous	Developer is waiting on screen
Monthly billing report consumed next morning	Batch	Overnight run, latency irrelevant
Pipeline step where next step is blocked	Synchronous	Downstream step cannot proceed

Each batch request gets a unique `custom_id` for tracking. Results arrive in a results file where you match responses to requests by their `custom_id`. Results do not come back in the same order as the input requests. Matching is always by `custom_id`, never by position.

EXAM TIP: When a question describes non-blocking work with acceptable latency (12+ hours, "needed by tomorrow morning," "generated overnight") and asks how to reduce per-token cost while keeping the same model, the answer is the Message Batches API. Distractors include: parallel real-time requests (concurrency doesn't reduce per-token cost), switching to a smaller model (changes model tier), or combining requests (different cost model).

The 50% Cost Savings Tradeoff

The Message Batches API charges 50% of standard synchronous API prices. The tradeoff is that there is no latency guarantee beyond the 24-hour processing window. Most batches complete in under an hour, but your system must be designed to handle the full window. Any SLA built on batch processing must absorb this uncertainty.

Batch processing does not support multi-turn tool loops within a single request. Agentic workflows that require Claude to call tools, receive results, and continue must use the synchronous API.

Prompt Caching with Pre-Warming

When running batch reviews with a shared system prompt (e.g., 8,000-token migration review guidelines), prompt caching can reduce costs further. However, batch requests may not execute in temporal proximity, causing cache entries to expire before later requests execute.

The solution is to add cache pre-warming requests with `max_tokens: 0` at the beginning of each batch. These requests cost nearly nothing but seed the cache, ensuring it's populated when the actual review requests start processing.

EXAM TIP: If a question describes low cache hit rates in batch processing concentrated on requests "processed later in the batch window", the answer is cache pre-warming. Not extending the cache TTL (different mechanism). Not splitting into sequential batches (adds latency). Not moving cache breakpoints to different content (wrong target).

References

- <https://docs.anthropic.com/en/docs/build-with-claude/batch-processing>
- <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>

M. Claude Code's Built-in Tools

Claude Code provides several built-in tools for interacting with the codebase. Understanding when to use each tool is important for both efficient exploration and correct exam answers. These tools are part of Claude Code's default system prompt which is why `--append-system-prompt` preserves them and `--system-prompt` strips them.

Anthropic's documentation categorizes the built-in tools as follows:

Category	Tools	What They Do
File operations	Read, Write, Edit	Read, modify, and create files
Search	Grep, Glob	Find content within files, find files by name pattern
Execution	Bash	Run shell commands, scripts, git operations

Read Tool

Loads file contents into Claude's context. Use when you need to understand what a file contains — its structure, logic, dependencies, and patterns. Read is non-destructive — it only adds information to context, never modifies files.

Write Tool

Creates a new file or overwrites an existing file entirely with new content. Use when creating new files or when the Edit tool can't find a unique match point. Write replaces the entire file content, so it should be used carefully on existing files.

Edit Tool

Modifies part of an existing file using an `old_string` → `new_string` replacement. The `old_string` must be unique in the file. Edit fails if the string appears multiple times or not at all. This is Claude's primary tool for making targeted changes to existing files.

Read → Write Fallback When Edit Fails

When a file has highly repetitive content (duplicate docstrings, repeated variable names, identical structural patterns), the Edit tool's `old_string` parameter may fail to find a unique match. The reliable fallback is to:

1. Read the file to load its contents.
2. Modify the content (add the new function, change the target section).
3. Write the updated file.

This fallback pattern is frequently tested because it represents a real limitation of the Edit tool's unique-string-matching approach. The Read → modify → Write pattern is less elegant than Edit but always works, regardless of content repetition.

EXAM TIP: When a question describes the Edit tool failing due to repetitive content in a 150-line file, and the developer needs to insert a function between two existing functions — the answer is Read → modify → Write. Not using an extremely long `old_string` (fragile). Not appending to the end with Bash heredoc (wrong placement). Not using `replace_all` (changes all instances, not targeted).

Grep Tool

Searches file contents for text patterns across the codebase. Use when you know what text to look for, an error message string, an import statement, a function name, or a variable reference.

Examples: finding all files that import `@company/auth`, locating where an error message "SYNC_CONFLICT: entity version mismatch" is defined, finding all callers of a specific function.

Glob Tool

Searches for files by name or path pattern. Use when you know the file naming convention but not the location finding all `cache*.py` files, locating `errors/` directories, or listing files matching `*.test.ts`.

EXAM TIP: Grep searches file contents; Glob searches file names. When a question asks to "find all files that import a specific package" use Grep (content search). When a question asks to "find files named cache something", use Glob (name search). This is a simple but frequently tested distinction.

Bash Tool

Executes arbitrary shell commands. Use for running tests, installing dependencies, checking git history, viewing file system information, or any operation available in the terminal.

Reference

- <https://docs.anthropic.com/en/docs/claude-code>

N. Codebase Exploration Strategies

The exam tests your ability to choose efficient exploration strategies when Claude needs to understand unfamiliar code. The right strategy depends on what you're trying to learn: system architecture, specific function usage, error origins, or test coverage priorities.

Each strategy pairs specific tools with a specific exploration goal. Top-down exploration uses Read to understand architecture. Finding callers uses Grep after mapping all names. Error message tracing uses Grep for distinctive text. Task decomposition uses Glob and Grep to map structure before planning.

Top-Down Exploration

When investigating a large subsystem (like a caching layer spanning 15 files and ~8,000 lines), the most efficient approach is:

1. Analyze imports and class hierarchies to identify the base cache class.

2. Read that file to understand the interface and core abstractions.
3. Trace specific implementations (like invalidation) from the base class outward.

This builds understanding from architecture down to details, managing context constraints by not loading all 15 files simultaneously. Loading all files would consume most of the context window, leaving little room for Claude to reason about the code.

EXAM TIP: When a question describes a large subsystem and asks for the "most effective next step for building understanding while managing context constraints" the answer is analyzing the architectural entry point first (base class, interface, or main module), not reading all files sequentially or grepping for keywords without structural context.

Finding All Callers Including Renamed Wrappers

When removing or renaming a function that's exposed through wrapper modules under different names (e.g., `calculateTax` in the library becomes `computeOrderTax` in the orders module), simply grepping for the original function name misses renamed exports.

The reliable strategy is:

1. Read the library and wrapper modules to identify all exposed names for the function.
2. Grep for each name across the codebase.

This two-step approach ensures no callers are missed, even when the function is re-exported under a different name.

Searching for Error Messages Across Services

When an unfamiliar error message appears in production logs and you don't know which of 12 services generates it, the most efficient approach is to Grep for the distinctive text from the error message (like "SYNC_CONFLICT" or "entity version mismatch"), then Read the matching files to understand context.

This is faster than: reading README files and exploring directories systematically (too slow), grepping for error handling imports (too broad), or searching for files in `errors/` directories (relies on naming conventions that may not exist).

Decomposing Open-Ended Tasks

When Claude receives an open-ended task like "add comprehensive tests to a legacy codebase with 200 files," the effective decomposition is:

1. Use Glob and Grep to map codebase structure.
2. Identify heavily-coupled modules and high-impact areas.
3. Create a prioritized plan for high-impact areas first.
4. Revise the plan as dependencies are discovered during implementation.

This is more effective than: starting alphabetically (no prioritization), reading all 200 files before writing any tests (wastes context), or creating a fixed schedule based on directory structure (ignores code complexity and business importance).

Exploration Goal	Primary Tool	Strategy
Understand a large subsystem's architecture	Read	Start from base class/interface, trace outward
Find all callers of a renamed function	Read + Grep	Read wrappers to get all names, Grep each name
Trace an error message to its source	Grep	Search for distinctive text across the codebase
Decompose a large open-ended task	Glob + Grep	Map structure, prioritize high-impact areas
Find files matching a naming pattern	Glob	Search by file name/path pattern
Find files importing a specific package	Grep	Search file contents for the import statement

Reference

- <https://docs.anthropic.com/en/docs/claude-code/best-practices>

O. The Self-Review Limitation

When Claude Code generates code during a development session and then reviews its own code in the same session, it retains context about its prior reasoning. This makes it less likely to question its own decisions — even when those decisions contain subtle issues. Understanding this limitation and knowing the correct architectural solution is a core Domain 3 exam topic.

This is not a prompt engineering problem that can be solved with better instructions. It is an architectural limitation rooted in how conversation context works. The same reasoning that produced the code is still present in the context when the review happens, creating a systematic bias toward confirming earlier decisions.

Context Retention Bias

Claude has already "convinced itself" that its approach is correct during generation. When asked to self-review, it re-evaluates through the same lens, confirming its earlier conclusions rather than questioning them. This is analogous to why human developers benefit from external code review. A fresh perspective catches things the original author's mental model filters out.

The exam tests this with scenarios where Claude's reasoning during generation shows it "considered these cases but concluded its approach was correct," yet a different team member or a separate CI review catches the issues. The fact that Claude considered and dismissed the issues during generation is what makes the self-review limitation severe; it's not that Claude didn't think about the edge cases, it's that it already decided they weren't problems.

Independent Claude Instance for Review

The solution is to have a second, independent Claude Code instance review the changes without seeing the generator's reasoning. This independent review has no prior context about why the code was written the way it was, so it evaluates the code purely on its merits.

The independent instance brings fresh attention to every line, every assumption, and every edge case. It has no prior conviction that the approach is correct, so it is equally likely to question each decision.

Concept	Best Used For	Key Benefit	Common Exam Trap
Self-review (same session)	Quick sanity checks	Low overhead	Trusted to catch subtle defects it generated
Independent review (separate instance)	Catching subtle defects in generated output	Fresh context, no generation bias	Overlooked in favor of "review your own work"

EXAM TIP: When a question describes subtle bugs that Claude considered but dismissed during generation, and the question asks which approach "directly addresses the root cause of this self-review limitation," the answer is an independent Claude instance. Distractors include extended thinking (same context, same biases), comprehensive test files (doesn't address the reasoning retention issue), and explicit self-review instructions (Claude already evaluated and concluded its approach was correct, asking it to self-review again doesn't add a fresh perspective).

Common Mistakes

- Asking Claude to "review more carefully" in the same session, the reasoning bias remains.
- Using extended thinking as a substitute for independent review, it operates in the same context with the same biases.
- Treating self-review as equivalent to external review, they are fundamentally different in what they can catch.

Reference

- <https://docs.anthropic.com/en/docs/claude-code/best-practices>

Domain 3 Services Appendix

Claude Code Configuration Reference

Mechanism	Location	Scope	Loads When	Enforcement
Root CLAUDE.md	Project root	All team members	Every session	Advisory
Subdirectory CLAUDE.md	Any project folder	All team members	When working in that directory	Advisory
User CLAUDE.md	~/.claude/CLAUDE.md	One developer	Every session across all projects	Advisory

Mechanism	Location	Scope	Loads When	Enforcement
@imports	Inside CLAUDE.md	Depends on containing file	When containing file loads	Advisory
.claude/rules/	.claude/rules/*.md	All team members	When working on matching file paths	Advisory
.claude/skills/	.claude/skills//SKILL.md	All team members	On demand via slash command	Advisory
~/claude/skills/	User home directory	One developer	On demand via slash command	Advisory
Hooks (settings.json)	.claude/settings.json	All team members	Automatically at lifecycle events	Deterministic
Hooks (local)	.claude/settings.local.json	One developer	Automatically at lifecycle events	Deterministic
permissions.deny	Settings	Configurable	Always enforced	Hard block
.mcp.json	Project root	All team members	Every session	N/A (server config)
~/claude.json	User home directory	One developer	Every session	N/A (server config)

CI/CD Flags Reference

Flag	Purpose	Enforced By
claude -p	Non-interactive piped/programmatic mode	Claude Code CLI
--max-turns N	Limit agentic iterations per invocation	Claude Code itself
--max-budget-usd X	Hard dollar cap per invocation	Claude Code itself
--system-prompt	Replace default system prompt entirely	Claude Code CLI
--append-system-prompt	Add to default system prompt	Claude Code CLI
--permission-mode dontAsk	Auto-deny unallowed permission requests	Claude Code CLI
--output-format json	Enforce JSON output	Claude Code CLI

Hook Events Reference

Event	Fires When	Common Use
SessionStart	Session begins or is restored	Inject context, check environment
PreToolUse	Before any tool executes	Block dangerous actions, validate inputs

Event	Fires When	Common Use
PostToolUse	After a tool completes successfully	Auto-format, lint, log
Stop	Claude finishes responding	Cleanup, telemetry
Notification	Claude sends a notification	Alert routing
SubagentStop	A subagent completes its task	Cleanup, validation

Built-in Tools Reference

Tool	Searches	Use For
Read	N/A (loads file)	Understanding file contents
Write	N/A (creates/overwrites file)	Creating new files, fallback when Edit fails
Edit	N/A (modifies file)	Targeted changes using unique string match
Grep	File contents	Finding text patterns, import statements, error messages
Glob	File names/paths	Finding files by naming pattern
Bash	N/A (executes commands)	Running tests, git operations, system commands

Message Batches API Reference

Property	Value
Cost savings	50% of standard synchronous API prices
Processing window	Up to 24 hours (most complete in under 1 hour)
Result matching	By custom_id, not by position (order is not preserved)
Multi-turn support	Not supported — single request/response only
Result availability	29 days after batch creation
Batch size limits	100,000 requests or 256 MB per batch
Use for	Latency-tolerant work: overnight reports, bulk analysis, audits
Not for	Blocking workflows, multi-turn tool loops, latency-sensitive work

MCP Configuration Reference

Element	Description
MCP tools	Callable functions exposed by MCP servers
MCP resources	Read-only content catalogs describing server data
MCP prompts	Server-defined prompts surfaced as /mcp__ slash commands
.mcp.json	Project-level configuration (shared, version-controlled)

Element	Description
~/.claude.json	User-level configuration (personal, not shared)

Domain 3: Claude Code Configuration & Workflows Output — Sample Questions

Question 1

You are using Claude Code to refactor a legacy application with interconnected modules. The workflow must ensure that updates do not break dependent components.

For refactoring tasks, Claude Code workflows should follow which practice?

1. Rely on no context
2. Process files simultaneously in parallel
3. Refactor files with manual verification
4. Process files in sequence with tests

Correct Answer: 4

Explanation:

Claude Code workflows support sequential task execution, iterative file updates, and integrated testing practices that help maintain consistency during refactoring operations. Anthropic documentation explains that Claude performs more effectively when tasks are broken into structured and manageable steps, allowing the model to reason through dependencies and apply changes in a controlled manner. Running tests after modifications also helps validate whether updates preserve the expected behavior of the application and its related components.

Processing files in sequence is an important practice when working with interconnected codebases because changes in one module can affect the behavior of another. By handling updates incrementally and validating results through testing, engineering teams can identify compatibility issues earlier in the workflow and reduce the likelihood of introducing regressions. This structured approach improves reliability by ensuring that dependencies, configurations, and shared logic remain functional throughout the refactoring process.

Sequential workflows combined with testing also create a more maintainable AI-assisted development pipeline. Instead of applying broad modifications across multiple files simultaneously, teams can isolate changes, verify outcomes, and refine updates before proceeding to the next stage. This improves traceability, strengthens quality assurance practices, and supports more dependable automation for large-scale software maintenance and modernization efforts.

Hence, the correct answer is: **Process files in sequence with tests.**

The option that says: *Rely on no context* is incorrect because Claude Code workflows typically perform better when sufficient project context, file relationships, and task instructions are provided. Anthropic documentation explains that structured context improves reasoning quality, consistency, and the model's ability to generate reliable outputs across connected tasks and dependent code components.

The option that says: *Refactor files with manual verification* is incorrect because Claude Code workflows are primarily designed to support automated and repeatable development processes that integrate validation steps, such as testing and iterative checks. Manual verification alone may not consistently detect dependency issues, regressions, or integration problems across interconnected modules in larger codebases.

The option that says: *Process files simultaneously in parallel* is incorrect because sequential task execution helps Claude maintain clearer reasoning across dependent updates and simply reduces the risk of conflicting modifications between interconnected files. Anthropic guidance emphasizes breaking complex tasks into manageable steps to improve reliability, traceability, and output quality during multi-stage workflows.

References:

- <https://code.claude.com/docs/en/best-practices>
- <https://code.claude.com/docs/en/common-workflows>

Question 2

Your CI pipeline needs Claude Code to produce structured JSON output that can be parsed and posted as inline PR comments.

Which CLI flags should you use?

1. `--output-format json` and `--json-schema` to enforce a specific output structure.
2. `--format json` and `--strict` to enforce JSON compliance.
3. `-p --json` to enable JSON mode alongside non-interactive mode.
4. `--structured-output` with a Pydantic schema file passed as an argument.

Correct Answer: 1

Explanation:

Claude Code offers several features that help automate workflows, especially in CI/CD pipelines. Among these features, the `--output-format json` and `--json-schema` flags are critical for ensuring that output is structured and adheres to a specific format. The `--output-format json` flag ensures that the output is in JSON, a widely used and machine-readable format. This makes it easy for automated systems to parse and process the data further, such as posting inline PR comments or updating issue trackers.

The `--json-schema` flag complements this by enforcing a specific structure for the output. This guarantees that the JSON data is not only in the correct format but also follows a predefined schema, ensuring consistency and avoiding errors in downstream processes. Using this flag is particularly useful in automated environments where data needs to meet exact specifications to work properly with other tools or systems.

In the context of a CI/CD pipeline, using both `--output-format json` and `--json-schema` ensures that Claude Code generates reliable, structured, and predictable output. This is vital for tasks like automated code reviews, where the output must be consistent to be parsed correctly and integrated into

the pipeline. These flags make the process smoother by eliminating the need for manual intervention and ensuring that the output meets the necessary requirements for automated systems to handle seamlessly.

Hence, the correct answer is: **--output-format json and --json-schema to enforce a specific output structure.**

The option that says: *--format json and --strict to enforce JSON compliance* is incorrect because it simply enforces JSON formatting but does not guarantee the output structure. While it ensures that the output is in JSON format, it does not define or enforce any specific structure, which is critical when parsing and posting inline PR comments in a predictable way.

The option that says: *-p --json to enable JSON mode alongside non-interactive mode* is incorrect because it primarily enables non-interactive mode and JSON output but does not provide any enforcement of a specific output structure. This flag is useful for non-interactive scenarios, but does not address the need for structured and schema-compliant JSON output, which is necessary for the scenario.

The option that says: *--structured-output with a Pydantic schema file passed as an argument* is incorrect because it refers to a non-existent flag in the Claude Code CLI. The correct flag for structuring output using a schema is **--json-schema**, not **--structured-output**. This means the suggested option is just not valid based on the official documentation.

References:

- <https://code.claude.com/docs/en/cli-reference>
- <https://code.claude.com/docs/en/headless>

References for Domain 3: Claude Code Configuration & Workflows Output

All links reference official Anthropic documentation.

Claude Code Overview

- <https://docs.anthropic.com/en/docs/claude-code>

Claude Code Best Practices

- <https://docs.anthropic.com/en/docs/claude-code/best-practices>

Claude Code Settings and Configuration

- <https://docs.anthropic.com/en/docs/claude-code/settings>

Claude Code Hooks

- <https://docs.anthropic.com/en/docs/claude-code/hooks>

Claude Code Memory and CLAUDE.md

- <https://docs.anthropic.com/en/docs/claude-code/memory>

Claude Code Skills

- <https://docs.anthropic.com/en/docs/claude-code/skills>

Claude Code Sub-Agents

- <https://docs.anthropic.com/en/docs/claude-code/sub-agents>

Claude Code Sessions

- <https://docs.anthropic.com/en/docs/claude-code/sessions>

Claude Code CI/CD Integration

- <https://docs.anthropic.com/en/docs/claude-code/github-actions>

Message Batches API

- <https://docs.anthropic.com/en/docs/build-with-claude/batch-processing>

Model Context Protocol

- <https://docs.anthropic.com/en/docs/agents-and-tools/mcp>

Prompt Caching

- <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>

Prompt Engineering

- <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>

CCA-F Official Exam Page

- <https://clau.de/CCAF>